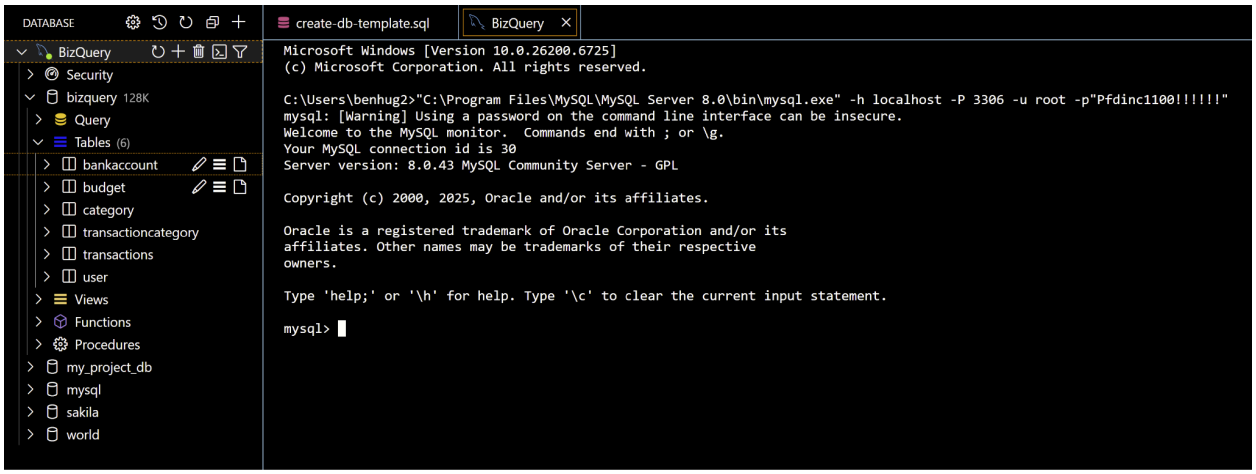
BizQuery Project Stage 3
Database Implementation and Indexing

By: Danny Guller, Viswanath Vadlamani,
Brandon Lee, and Ben Hug

All of this code is available in the "Stage 3 Queries.sql" file in the /doc folder on GitHub.

Part 1

MySQL Connection:



This screenshot is proof of connection to the database and MySQL. You can see the tables and database implemented on the left side of the screenshot as well.

DDL Commands for the Tables:

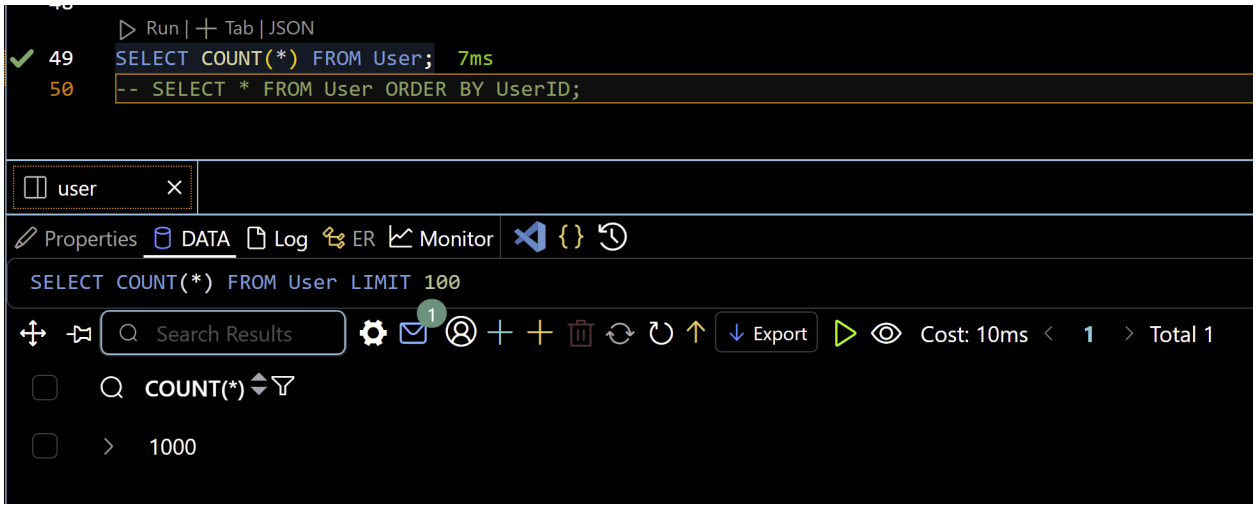
```sql
CREATE TABLE User (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL,
    CreatedDate DATE NOT NULL
);
CREATE TABLE BankAccount (
    AccountNumber VARCHAR(20) PRIMARY KEY,
    UserID INT NOT NULL,
    RoutingNumber VARCHAR(20),
    AccountType VARCHAR(20),
    AccountOpenedDate DATE,
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);
CREATE TABLE Category (
    CategoryID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    CategoryName VARCHAR(50) NOT NULL
);
CREATE TABLE Transactions (
    TransactionID INT AUTO_INCREMENT PRIMARY KEY,
    AccountNumber VARCHAR(20) NOT NULL,
    Date DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    PaymentMethod VARCHAR(30),
    Notes VARCHAR(255),
    FOREIGN KEY (AccountNumber) REFERENCES BankAccount(AccountNumber)
);
CREATE TABLE TransactionCategory (
    TransactionID INT NOT NULL,
    CategoryID INT NOT NULL,
    PRIMARY KEY (TransactionID, CategoryID),
    FOREIGN KEY (TransactionID) REFERENCES Transactions(TransactionID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
CREATE TABLE Budget (
    BudgetID INT AUTO_INCREMENT PRIMARY KEY,
    UserID INT NOT NULL,
    CategoryID INT NOT NULL,
    Duration VARCHAR(20),
    Max_Value DECIMAL(10,2),
    ConstraintName VARCHAR(50),
    FOREIGN KEY (UserID) REFERENCES User(UserID),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
```

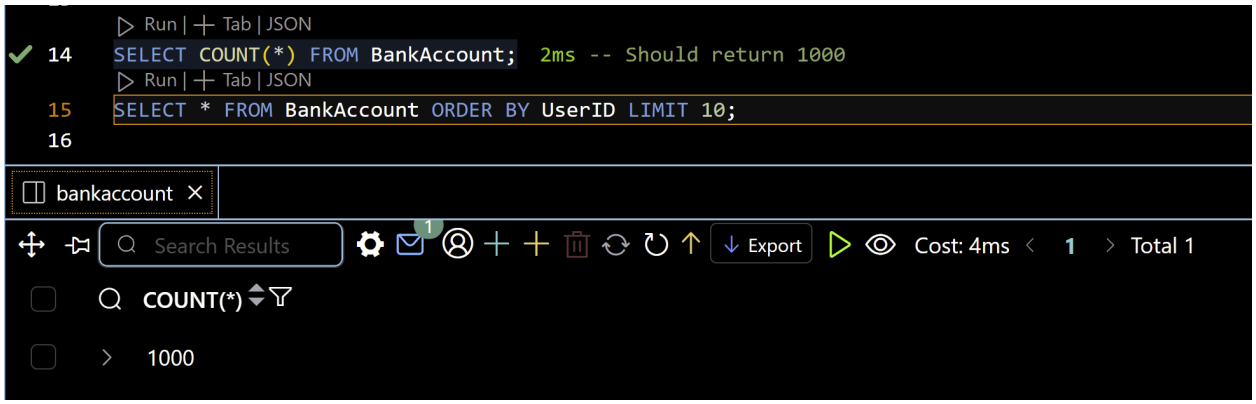
These are the SQL commands that we used to implement the database. They contain all of the necessary commands to replicate the database.

Insertion of 1000 Rows into User, Transactions, BankAccount:

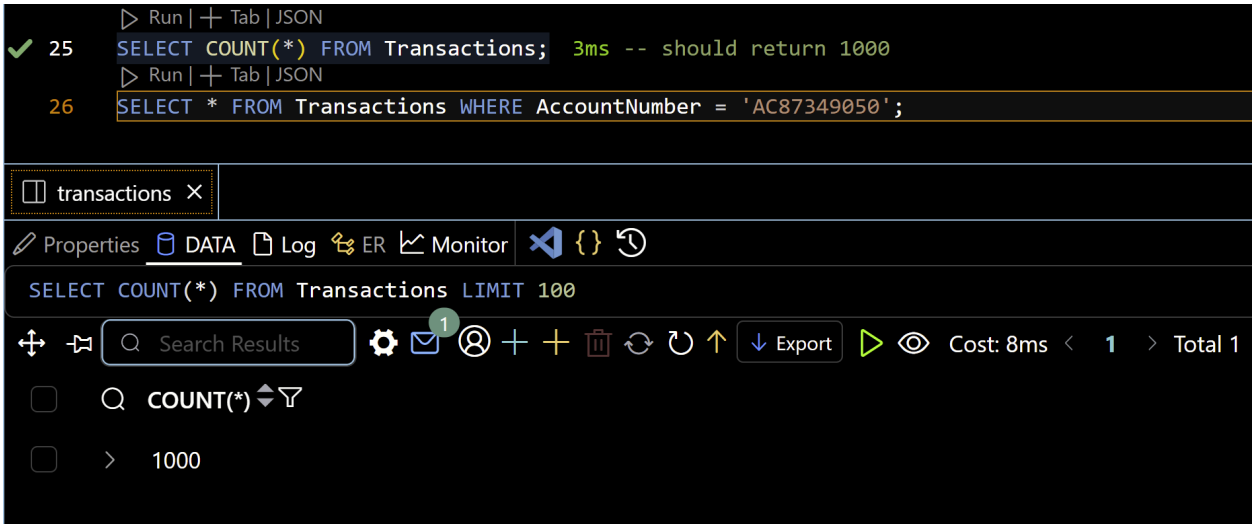
User:



BankAccount:



Transactions:

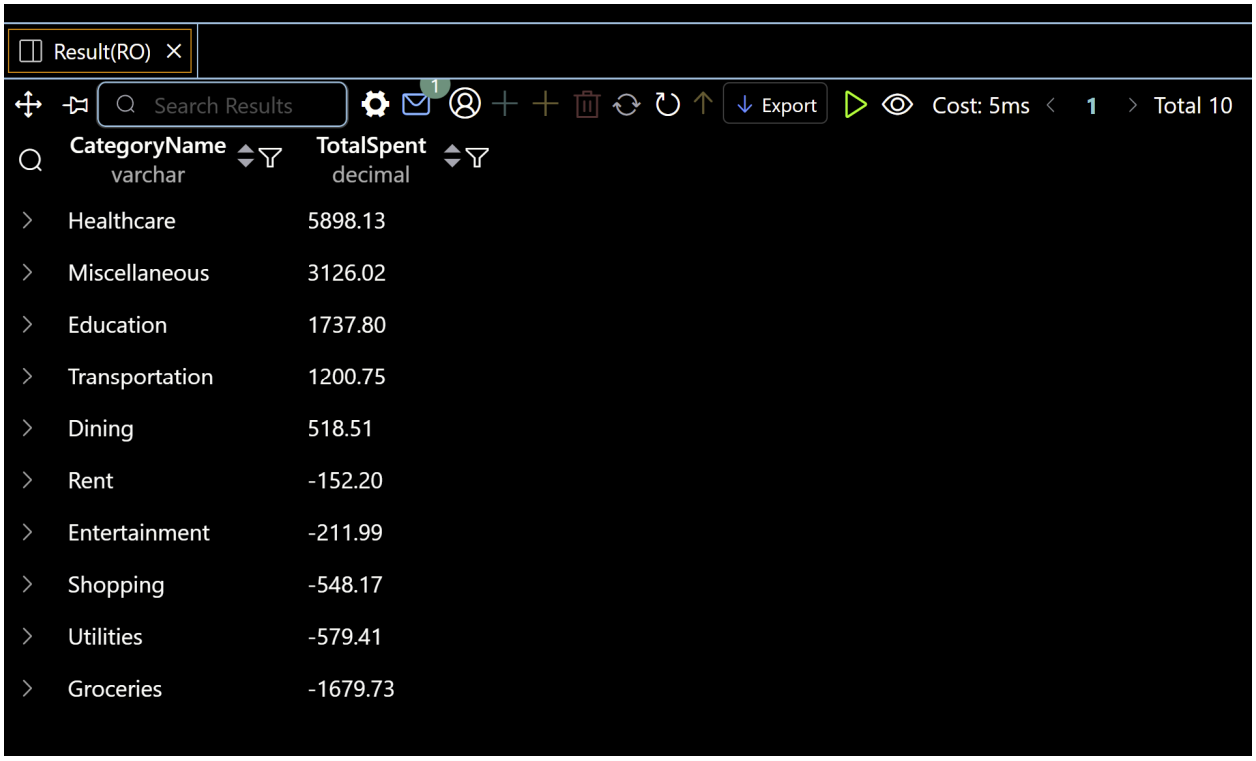


The count queries above show that 1,000 records were inserted into each of the User, BankAccount, and Transactions tables. The User and BankAccount tables were filled with different Users and Accounts for those users. The Transactions table was filled with transaction data for a singular user, UserID 256.

Advanced SQL Queries:

Query 1: Monthly Spending by Category for One User

```sql
SELECT
    c.CategoryName,
    SUM(t.Amount) AS TotalSpent
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
  AND t.Date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY c.CategoryName
ORDER BY TotalSpent DESC;
```



There are only 10 categories, therefore we see 10 results in our query. The query returns the amount of money spent per month in each respective category. The negative numbers represent the amount received in that category. The positive numbers represent the amount spent in that category.

Query 2: User Exceeding Budget Limits

```
SELECT
    u.Username,
    c.CategoryName,
    bu.Duration,
    SUM(t.Amount) AS TotalSpent,
    bu.Max_Value AS BudgetLimit
FROM User u
JOIN BankAccount ba
    ON u.UserID = ba.UserID
JOIN Transactions t
    ON ba.AccountNumber = t.AccountNumber
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN Budget bu
    ON bu.UserID = u.UserID AND bu.CategoryID = c.CategoryID
```

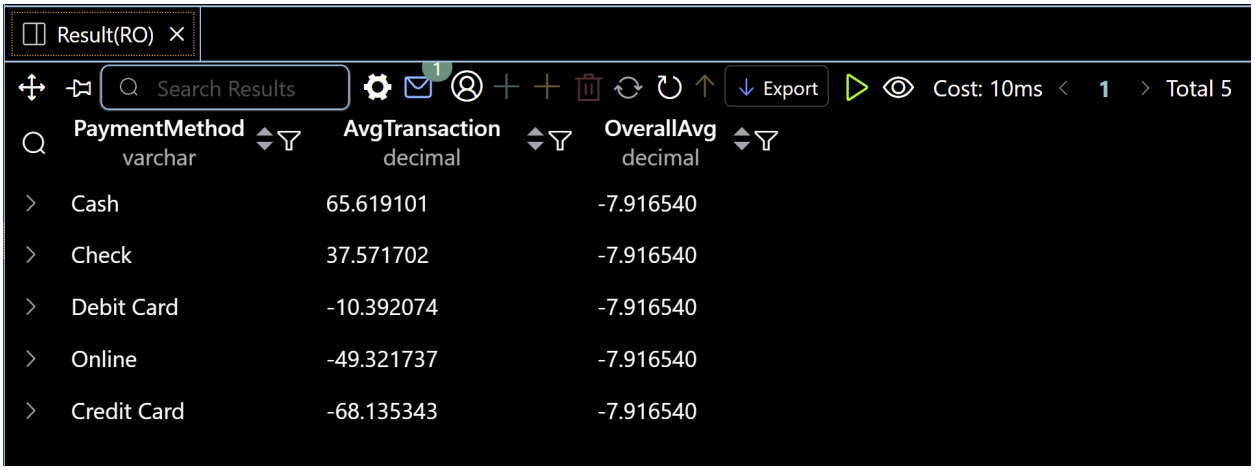
```
WHERE u.UserID = 256 AND t.Amount > 0
GROUP BY u.Username, c.CategoryName, bu.Duration, bu.Max_Value
HAVING SUM(t.Amount) > bu.Max_Value;
```

Username varchar	CategoryName varchar	Duration varchar	TotalSpent decimal	BudgetLimit decimal
sandra.brown203	Groceries	Monthly	21422.50	500.00
sandra.brown203	Rent	Monthly	22777.60	1200.00
sandra.brown203	Utilities	Monthly	31589.15	300.00
sandra.brown203	Dining	Monthly	21551.00	250.00
sandra.brown203	Entertainment	Monthly	20001.74	200.00

The query looks at a single user and whether they have gone over a certain amount inside of their budget limit. This query shows us that the user went over the budget in 5 of her budgets for the current month.

Query 3: Average Transaction Amount per Payment Method

```
SELECT t.PaymentMethod, AVG(t.Amount) AS AvgTransaction,
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     JOIN BankAccount b2 ON t2.AccountNumber = b2.AccountNumber
     WHERE b2.UserID = 256) AS OverallAvg
FROM Transactions t
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
GROUP BY t.PaymentMethod
ORDER BY AvgTransaction DESC;
```



There are 5 different PaymentMethods, therefore we see 5 different averages and transaction methods returned. This shows that the user is gaining money on average in 2 of her transaction types and losing money in 3 of the transaction types. This could be due to steady income the user is receiving or other factors.
Note: Positive averages indicate spending (debits), while negative averages represent income (credits) received by the user.

Query 4: Last Few Transactions per Category

```
SELECT
    t.TransactionID,
    t.Date AS TransactionDate,
    t.Amount,
    t.PaymentMethod,
    c.CategoryName
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
```

```
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
LEFT JOIN Transactions t2
    JOIN TransactionCategory tc2
        ON t2.TransactionID = tc2.TransactionID
    ON tc.CategoryID = tc2.CategoryID
    AND b.UserID = 256 AND t.Date < t2.Date AND t.AccountNumber =
t2.AccountNumber
WHERE b.UserID = 256
GROUP BY t.TransactionID, c.CategoryName, t.Date, t.Amount,
t.PaymentMethod
HAVING COUNT(t2.TransactionID) < 5
ORDER BY c.CategoryName, t.Date DESC
LIMIT 15;
```

TransactionID int	TransactionDate date	Amount decimal	PaymentMethod varchar	CategoryName varchar
330	2025-10-11	-217.12	Online	Dining
254	2025-10-09	-725.68	Check	Dining
750	2025-10-08	389.53	Debit Card	Dining
398	2025-10-06	309.56	Debit Card	Dining
476	2025-10-06	490.65	Online	Dining
241	2025-10-12	-232.55	Online	Education
86	2025-10-10	-448.31	Debit Card	Education
384	2025-10-10	-815.88	Debit Card	Education
67	2025-10-09	-346.74	Check	Education
47	2025-10-09	468.56	Check	Education
564	2025-10-09	701.02	Credit Card	Education
399	2025-10-11	-697.21	Check	Entertainment
731	2025-10-11	359.00	Debit Card	Entertainment
215	2025-10-10	-722.65	Check	Entertainment
487	2025-10-06	985.58	Cash	Entertainment

This query returns the last few transactions per category. It is limited to 15 transactions. As you can see, the Dining category has had 5 transactions since October 6, 2025.

Part 2

Query 1: Monthly Spending by Category for One User

```
EXPLAIN ANALYZE
SELECT
    c.CategoryName,
    SUM(t.Amount) AS TotalSpent
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
  AND t.Date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY c.CategoryName
ORDER BY TotalSpent DESC;
```

Output:

```
EXPLAIN -> Sort: TotalSpent DESC (actual time=4.4..4.4 rows=10 loops=1)
    -> Table scan on <temporary> (actual time=4.31..4.32 rows=10 loops=1)
    -> Aggregate using temporary table (actual time=4.31..4.31 rows=10 loops=1)
    -> Nested loop inner join (cost=336 rows=333) (actual time=0.22..4.02 rows=101 loops=1)
    -> Nested loop inner join (cost=219 rows=333) (actual time=0.206..3.58 rows=101 loops=1)
```

```
I   -> Filter: (t.`Date` >= <cache>(date_format(curdate(),'%Y-%m-01'))) (cost=102 rows=333)
N   (actual time=0.185..2.56 rows=101 loops=1)
    -> Inner hash join (t.AccountNumber = b.AccountNumber) (cost=102 rows=333) (actual
    time=0.156..1.89 rows=1000 loops=1)
    -> Table scan on t (cost=34.8 rows=1000) (actual time=0.0722..1.1 rows=1000 loops=1)
    -> Hash -> Covering index lookup on b using UserID (UserID=256) (cost=0.725 rows=1) (actual
    time=0.0385..0.0455 rows=1 loops=1)
    -> Covering index lookup on tc using PRIMARY (TransactionID=t.TransactionID) (cost=0.25
    rows=1) (actual time=0.00766..0.00961 rows=1 loops=101)
    -> Single-row index lookup on c using PRIMARY (CategoryID=tc.CategoryID) (cost=0.25
    rows=1) (actual time=0.00375..0.00382 rows=1 loops=101)
```

Index #1:

```sql
CREATE INDEX idx_transactions_date_acc
ON Transactions (Date, AccountNumber);

EXPLAIN ANALYZE
SELECT
    c.CategoryName,
    SUM(t.Amount) AS TotalSpent
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
  AND t.Date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY c.CategoryName
ORDER BY TotalSpent DESC;
```

Output:

```
E   -> Sort: TotalSpent DESC (actual time=1.34..1.34 rows=10 loops=1)
X   -> Table scan on <temporary> (actual time=1.29..1.3 rows=10 loops=1)
P   -> Aggregate using temporary table (actual time=1.29..1.29 rows=10 loops=1)
L   -> Nested loop inner join (cost=207 rows=100) (actual time=0.252..1.12 rows=101 loops=1)
A   -> Nested loop inner join (cost=172 rows=100) (actual time=0.245..0.93 rows=101 loops=1)
I   -> Nested loop inner join (cost=136 rows=100) (actual time=0.235..0.534 rows=101 loops=1)
N   -> Covering index lookup on b using UserID (UserID=256) (cost=0.725 rows=1) (actual
    time=0.0167..0.0184 rows=1 loops=1)
```

```
    -> Filter: ((t.AccountNumber = b.AccountNumber) and (t.`Date` >=
    <cache>(date_format(curdate(),'%Y-%m-01')))) (cost=45.2 rows=100) (actual time=0.216..0.502
    rows=101 loops=1)
    -> Index range scan on t (re-planned for each iteration) (cost=45.2 rows=1000) (actual
    time=0.211..0.398 rows=101 loops=1)
    -> Covering index lookup on tc using PRIMARY (TransactionID=t.TransactionID) (cost=0.251
    rows=1) (actual time=0.00246..0.0035 rows=1 loops=101)
    -> Single-row index lookup on c using PRIMARY (CategoryID=tc.CategoryID) (cost=0.251
    rows=1) (actual time=0.0014..0.00145 rows=1 loops=101)
```

This index type showed a major improvement in the cost. The cost of the original query was 336 and it was reduced to 207. The index filtered transactions by Date and AccountNumber at the same time. This avoided the full table scan on Transactions and reduced the number of loops needed.

Index #2:

```sql
CREATE INDEX i_bankaccount_user
ON BankAccount (UserID);

EXPLAIN ANALYZE
SELECT
    c.CategoryName,
    SUM(t.Amount) AS TotalSpent
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
  AND t.Date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY c.CategoryName
ORDER BY TotalSpent DESC;
```

Output:

```
E   -> Sort: TotalSpent DESC (actual time=1.77..1.77 rows=10 loops=1)
X   -> Table scan on <temporary> (actual time=1.69..1.72 rows=10 loops=1)
P   -> Aggregate using temporary table (actual time=1.69..1.69 rows=10 loops=1)
```

```
L   -> Nested loop inner join (cost=207 rows=100) (actual time=0.293..1.44 rows=101 loops=1)
A   -> Nested loop inner join (cost=172 rows=100) (actual time=0.284..1.22 rows=101 loops=1)
I   -> Nested loop inner join (cost=136 rows=100) (actual time=0.272..0.607 rows=101 loops=1)
N   -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1)
    (actual time=0.0178..0.0213 rows=1 loops=1)
    -> Filter: ((t.AccountNumber = b.AccountNumber) and (t.`Date` >=
    <cache>(date_format(curdate(),'%Y-%m-01')))) (cost=45.2 rows=100) (actual time=0.252..0.57
    rows=101 loops=1)
    -> Index range scan on t (re-planned for each iteration) (cost=45.2 rows=1000) (actual
    time=0.246..0.455 rows=101 loops=1)
    -> Covering index lookup on tc using PRIMARY (TransactionID=t.TransactionID) (cost=0.251
    rows=1) (actual time=0.00454..0.00568 rows=1 loops=101)
    -> Single-row index lookup on c using PRIMARY (CategoryID=tc.CategoryID) (cost=0.251
    rows=1) (actual time=0.00169..0.00176 rows=1 loops=101)
```

This index showed a similar improvement to the first index, improving the cost from 336 to 207. The index helped look up the user's bank account quickly. The Transactions scan itself shouldn't have been affected.

Index #3:

```sql
CREATE INDEX i_tc_categoryid ON TransactionCategory(CategoryID);

EXPLAIN ANALYZE
SELECT
    c.CategoryName,
    SUM(t.Amount) AS TotalSpent
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
  AND t.Date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY c.CategoryName
ORDER BY TotalSpent DESC;
```

Output:

```
E   -> Sort: TotalSpent DESC (actual time=1.16..1.16 rows=10 loops=1)
X   -> Table scan on <temporary> (actual time=1.13..1.13 rows=10 loops=1)
P   -> Aggregate using temporary table (actual time=1.13..1.13 rows=10 loops=1)
L   -> Nested loop inner join (cost=207 rows=100) (actual time=0.192..0.96 rows=101 loops=1)
```

```
A   -> Nested loop inner join (cost=172 rows=100) (actual time=0.187..0.794 rows=101 loops=1)
I   -> Nested loop inner join (cost=136 rows=100) (actual time=0.177..0.431 rows=101 loops=1)
N   -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1)
    (actual time=0.0286..0.0299 rows=1 loops=1)
    -> Filter: ((t.AccountNumber = b.AccountNumber) and (t.`Date` >=
    <cache>(date_format(curdate(),'%Y-%m-01')))) (cost=45.2 rows=100) (actual time=0.147..0.394
    rows=101 loops=1)
    -> Index range scan on t (re-planned for each iteration) (cost=45.2 rows=1000) (actual
    time=0.143..0.29 rows=101 loops=1)
    -> Covering index lookup on tc using PRIMARY (TransactionID=t.TransactionID) (cost=0.251
    rows=1) (actual time=0.00244..0.00326 rows=1 loops=101)
     -> Single-row index lookup on c using PRIMARY (CategoryID=tc.CategoryID) (cost=0.251
    rows=1) (actual time=0.00121..0.00125 rows=1 loops=101)
```

This index barely improved performance, and the time needed consistently went down around 1.1 milliseconds. The cost remained roughly the same. This helps join the TransactionCategory and Category, but the lookup is not all that intensive in the first place.

Final Choice: Index #1 is the best index because it directly supports the joining and filtering happening in the query. It also gave us the strongest cost and time reduction.

Query 2: User Exceeding Budget Limits

```sql
EXPLAIN ANALYZE
SELECT
    u.Username,
    c.CategoryName,
    bu.Duration,
    SUM(t.Amount) AS TotalSpent,
    bu.Max_Value AS BudgetLimit
FROM User u
JOIN BankAccount ba
    ON u.UserID = ba.UserID
JOIN Transactions t
    ON ba.AccountNumber = t.AccountNumber
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN Budget bu
    ON bu.UserID = u.UserID AND bu.CategoryID = c.CategoryID
WHERE u.UserID = 256 AND t.Amount > 0
GROUP BY u.Username, c.CategoryName, bu.Duration, bu.Max_Value
HAVING SUM(t.Amount) > bu.Max_Value;
```

Output:

```
EXPLAIN -> Filter: (`sum(t.Amount)` > bu.Max_Value) (actual time=3.12..3.13 rows=5 loops=1)
    -> Table scan on <temporary> (actual time=3.12..3.12 rows=5 loops=1)
    -> Aggregate using temporary table (actual time=3.12..3.12 rows=5 loops=1)
    -> Nested loop inner join (cost=235 rows=167) (actual time=0.707..2.57 rows=240 loops=1)
    -> Nested loop inner join (cost=59.7 rows=500) (actual time=0.684..0.978 rows=492 loops=1) ->
    Nested loop inner join (cost=3.98 rows=5) (actual time=0.403..0.431 rows=5 loops=1)
    -> Inner hash join (no condition) (cost=2.23 rows=5) (actual time=0.373..0.386 rows=5 loops=1)
    -> Filter: (bu.UserID = 256) (cost=1.5 rows=5) (actual time=0.318..0.327 rows=5 loops=1)
    -> Table scan on bu (cost=1.5 rows=5) (actual time=0.316..0.323 rows=5 loops=1)
    -> Hash -> Covering index lookup on ba using UserID (UserID=256) (cost=0.725 rows=1)
    (actual time=0.0291..0.0342 rows=1 loops=1)
    -> Single-row index lookup on c using PRIMARY (CategoryID=bu.CategoryID) (cost=0.27
    rows=1) (actual time=0.00826..0.00834 rows=1 loops=5)
    -> Covering index lookup on tc using CategoryID (CategoryID=bu.CategoryID) (cost=3.14
    rows=100) (actual time=0.0728..0.0995 rows=98.4 loops=5)
    -> Filter: ((t.AccountNumber = ba.AccountNumber) and (t.Amount > 0.00)) (cost=0.25
    rows=0.333) (actual time=0.00292..0.00299 rows=0.488 loops=492)
    -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25
    rows=1) (actual time=0.00214..0.00219 rows=1 loops=492)
```

Index #1:

```sql
CREATE INDEX i_ba_userid ON BankAccount(UserID);

EXPLAIN ANALYZE
SELECT
    u.Username,
    c.CategoryName,
    bu.Duration,
    SUM(t.Amount) AS TotalSpent,
    bu.Max_Value AS BudgetLimit
FROM User u
JOIN BankAccount ba
    ON u.UserID = ba.UserID
JOIN Transactions t
    ON ba.AccountNumber = t.AccountNumber
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN Budget bu
```

```
    ON bu.UserID = u.UserID AND bu.CategoryID = c.CategoryID
WHERE u.UserID = 256 AND t.Amount > 0
GROUP BY u.Username, c.CategoryName, bu.Duration, bu.Max_Value
HAVING SUM(t.Amount) > bu.Max_Value;
```

Output:

```
EXPLAIN -> Filter: (`sum(t.Amount)` > bu.Max_Value) (actual time=2.57..2.57 rows=5 loops=1)
    -> Table scan on <temporary> (actual time=2.57..2.57 rows=5 loops=1)
    -> Aggregate using temporary table (actual time=2.56..2.56 rows=5 loops=1)
    -> Nested loop inner join (cost=230 rows=167) (actual time=0.103..1.95 rows=240 loops=1)
    -> Nested loop inner join (cost=54.9 rows=500) (actual time=0.0874..0.419 rows=492 loops=1)
    -> Nested loop inner join (cost=3.48 rows=5) (actual time=0.0564..0.0751 rows=5 loops=1)
    -> Nested loop inner join (cost=1.73 rows=5) (actual time=0.0443..0.0513 rows=5 loops=1)
    -> Covering index lookup on ba using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0124..0.0142 rows=1 loops=1)
    -> Index lookup on bu using UserID (UserID=256) (cost=1 rows=5) (actual time=0.0302..0.0348 rows=5 loops=1)
    -> Single-row index lookup on c using PRIMARY (CategoryID=bu.CategoryID) (cost=0.27 rows=1) (actual time=0.00418..0.00424 rows=1 loops=5)
    -> Covering index lookup on tc using i_tc_categoryid (CategoryID=bu.CategoryID) (cost=2.29 rows=100) (actual time=0.0262..0.058 rows=98.4 loops=5)
    -> Filter: ((t.AccountNumber = ba.AccountNumber) and (t.Amount > 0.00)) (cost=0.25 rows=0.333) (actual time=0.00269..0.00282 rows=0.488 loops=492)
    -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25 rows=1) (actual time=0.00179..0.00184 rows=1 loops=492)
```

This index was a strong improvement from the original query. The cost was reduced from 235 to 230. The cost was reduced for joining User and BankAccount and made the UserID lookups efficient.

Index #2:

```
CREATE INDEX i_t_an ON Transactions(AccountNumber);

EXPLAIN ANALYZE
SELECT
    u.Username,
    c.CategoryName,
    bu.Duration,
    SUM(t.Amount) AS TotalSpent,
    bu.Max_Value AS BudgetLimit
FROM User u
JOIN BankAccount ba
```

```
    ON u.UserID = ba.UserID
JOIN Transactions t
    ON ba.AccountNumber = t.AccountNumber
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN Budget bu
    ON bu.UserID = u.UserID AND bu.CategoryID = c.CategoryID
WHERE u.UserID = 256 AND t.Amount > 0
GROUP BY u.Username, c.CategoryName, bu.Duration, bu.Max_Value
HAVING SUM(t.Amount) > bu.Max_Value;
```

Output:

```
E   -> Filter: (`sum(t.Amount)` > bu.Max_Value) (actual time=2.37..2.38 rows=5 loops=1)
X   -> Table scan on <temporary> (actual time=2.37..2.37 rows=5 loops=1)
P   -> Aggregate using temporary table (actual time=2.37..2.37 rows=5 loops=1)
L   -> Nested loop inner join (cost=230 rows=167) (actual time=0.09..1.82 rows=240 loops=1)
A   -> Nested loop inner join (cost=54.9 rows=500) (actual time=0.0767..0.391 rows=492 loops=1)
I   -> Nested loop inner join (cost=3.48 rows=5) (actual time=0.0464..0.064 rows=5 loops=1)
N   -> Nested loop inner join (cost=1.73 rows=5) (actual time=0.0354..0.0424 rows=5 loops=1)
    -> Covering index lookup on ba using i_bankaccount_user (UserID=256) (cost=0.725 rows=1)
    (actual time=0.0117..0.0135 rows=1 loops=1)
    -> Index lookup on bu using UserID (UserID=256) (cost=1 rows=5) (actual time=0.0221..0.0264
    rows=5 loops=1)
    -> Single-row index lookup on c using PRIMARY (CategoryID=bu.CategoryID) (cost=0.27
    rows=1) (actual time=0.00374..0.0038 rows=1 loops=5)
    -> Covering index lookup on tc using i_tc_categoryid (CategoryID=bu.CategoryID) (cost=2.29
    rows=100) (actual time=0.0249..0.0546 rows=98.4 loops=5)
    -> Filter: ((t.AccountNumber = ba.AccountNumber) and (t.Amount > 0.00)) (cost=0.25
    rows=0.333) (actual time=0.00254..0.00262 rows=0.488 loops=492)
    -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25
    rows=1) (actual time=0.00163..0.00168 rows=1 loops=492)
```

The index had similar improvements to the first index, with the time being reduced by 2 milliseconds consistently. The joins between BankAccount and Transactions are optimized, but this was not necessarily needed.

Index #3:

```
CREATE INDEX i_amount ON Transactions(Amount);


EXPLAIN ANALYZE
```

```sql
SELECT
    u.Username,
    c.CategoryName,
    bu.Duration,
    SUM(t.Amount) AS TotalSpent,
    bu.Max_Value AS BudgetLimit
FROM User u
JOIN BankAccount ba
    ON u.UserID = ba.UserID
JOIN Transactions t
    ON ba.AccountNumber = t.AccountNumber
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN Budget bu
    ON bu.UserID = u.UserID AND bu.CategoryID = c.CategoryID
WHERE u.UserID = 256 AND t.Amount > 0
GROUP BY u.Username, c.CategoryName, bu.Duration, bu.Max_Value
HAVING SUM(t.Amount) > bu.Max_Value;
```

Output:

```
EXPLAIN -> Filter: (`sum(t.Amount)` > bu.Max_Value) (actual time=2.99..2.99 rows=5 loops=1)
    -> Table scan on <temporary> (actual time=2.98..2.99 rows=5 loops=1)
    -> Aggregate using temporary table (actual time=2.98..2.98 rows=5 loops=1)
    -> Nested loop inner join (cost=230 rows=244) (actual time=0.0913..2.42 rows=240 loops=1)
    -> Nested loop inner join (cost=54.9 rows=500) (actual time=0.0777..0.408 rows=492 loops=1)
    -> Nested loop inner join (cost=3.48 rows=5) (actual time=0.0468..0.0694 rows=5 loops=1)
    -> Nested loop inner join (cost=1.73 rows=5) (actual time=0.036..0.0454 rows=5 loops=1)
    -> Covering index lookup on ba using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0116..0.0143 rows=1 loops=1)
    -> Index lookup on bu using UserID (UserID=256) (cost=1 rows=5) (actual time=0.023..0.0286 rows=5 loops=1)
    -> Single-row index lookup on c using PRIMARY (CategoryID=bu.CategoryID) (cost=0.27 rows=1) (actual time=0.00424..0.00432 rows=1 loops=5)
    -> Covering index lookup on tc using i_tc_categoryid (CategoryID=bu.CategoryID) (cost=2.29 rows=100) (actual time=0.0259..0.0564 rows=98.4 loops=5)
    -> Filter: ((t.AccountNumber = ba.AccountNumber) and (t.Amount > 0.00)) (cost=0.25 rows=0.489) (actual time=0.00373..0.00381 rows=0.488 loops=492)
    -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25 rows=1) (actual time=0.00183..0.00188 rows=1 loops=492)
```

The performance for this index actually got worse. The cost stayed the same but the time increased by a millisecond consistently. While the Amount is greater than 0, every row qualifies under the condition in the query. Therefore, this index on Amount is inefficient.

Final Choice: Index #1 is the best index due to the improvement of the user joins early in the query. This also reduced the number of nested loops needed before the aggregation call.

Query 3: Average Transaction Amount per Payment Method

```
EXPLAIN ANALYZE
SELECT t.PaymentMethod, AVG(t.Amount) AS AvgTransaction,
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     JOIN BankAccount b2 ON t2.AccountNumber = b2.AccountNumber
     WHERE b2.UserID = 256) AS OverallAvg
FROM Transactions t
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
GROUP BY t.PaymentMethod
ORDER BY AvgTransaction DESC;
```

Output:

```
EXPLAIN -> Sort: AvgTransaction DESC (actual time=2.86..2.87 rows=5 loops=1)
    -> Table scan on <temporary> (actual time=2.82..2.83 rows=5 loops=1)
    -> Aggregate using temporary table (actual time=2.82..2.82 rows=5 loops=1)
    -> Inner hash join (t.AccountNumber = b.AccountNumber) (cost=102 rows=1000) (actual
time=0.175..1.29 rows=1000 loops=1)
    -> Table scan on t (cost=102 rows=1000) (actual time=0.0623..0.622 rows=1000 loops=1)
    -> Hash -> Covering index lookup on b using UserID (UserID=256) (cost=0.725 rows=1) (actual
time=0.0591..0.0694 rows=1 loops=1)
    -> Select #2 (subquery in projection; run only once)
    -> Aggregate: avg(t2.Amount) (cost=202 rows=1) (actual time=1.16..1.16 rows=1 loops=1)
    -> Inner hash join (t2.AccountNumber = b2.AccountNumber) (cost=102 rows=1000) (actual
time=0.0689..0.916 rows=1000 loops=1)
    -> Table scan on t2 (cost=102 rows=1000) (actual time=0.0334..0.455 rows=1000 loops=1)
    -> Hash -> Covering index lookup on b2 using UserID (UserID=256) (cost=0.725 rows=1)
(actual time=0.0208..0.0227 rows=1 loops=1)
```

Index #1:

```
CREATE INDEX i_tran_ac ON Transactions(AccountNumber);
```

```
EXPLAIN ANALYZE
SELECT t.PaymentMethod, AVG(t.Amount) AS AvgTransaction,
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     JOIN BankAccount b2 ON t2.AccountNumber = b2.AccountNumber
     WHERE b2.UserID = 256) AS OverallAvg
FROM Transactions t
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
GROUP BY t.PaymentMethod
ORDER BY AvgTransaction DESC;
```

Output:

```
EXPLAIN
-> Sort: AvgTransaction DESC (actual time=2.33..2.33 rows=5 loops=1)
-> Table scan on <temporary> (actual time=2.3..2.31 rows=5 loops=1)
-> Aggregate using temporary table (actual time=2.3..2.3 rows=5 loops=1)
-> Inner hash join (t.AccountNumber = b.AccountNumber) (cost=102 rows=1000) (actual time=0.116..1.12 rows=1000 loops=1)
-> Table scan on t (cost=102 rows=1000) (actual time=0.0561..0.632 rows=1000 loops=1)
-> Hash -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0293..0.0349 rows=1 loops=1)
-> Select #2 (subquery in projection; run only once)
-> Aggregate: avg(t2.Amount) (cost=202 rows=1) (actual time=1.15..1.15 rows=1 loops=1)
-> Inner hash join (t2.AccountNumber = b2.AccountNumber) (cost=102 rows=1000) (actual time=0.0565..0.911 rows=1000 loops=1)
-> Table scan on t2 (cost=102 rows=1000) (actual time=0.0334..0.471 rows=1000 loops=1)
-> Hash -> Covering index lookup on b2 using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0106..0.0122 rows=1 loops=1)
```

Index #1 slightly improves the cost and improves the time by roughly 0.5 milliseconds consistently. This index helps the join Transactions and BankAccount in the main query and the subquery. This removes some redundant table scans.

Index #2:

```
CREATE INDEX i_tp ON Transactions(PaymentMethod);

EXPLAIN ANALYZE
SELECT t.PaymentMethod, AVG(t.Amount) AS AvgTransaction,
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     JOIN BankAccount b2 ON t2.AccountNumber = b2.AccountNumber
```

```
     WHERE b2.UserID = 256) AS OverallAvg
FROM Transactions t
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
GROUP BY t.PaymentMethod
ORDER BY AvgTransaction DESC;
```

Output:

```
EXPLAIN -> Sort: AvgTransaction DESC (actual time=2.33..2.33 rows=5 loops=1)
-> Table scan on <temporary> (actual time=2.29..2.29 rows=5 loops=1)
-> Aggregate using temporary table (actual time=2.29..2.29 rows=5 loops=1)
-> Inner hash join (t.AccountNumber = b.AccountNumber) (cost=102 rows=1000) (actual
time=0.116..1.17 rows=1000 loops=1)
-> Table scan on t (cost=102 rows=1000) (actual time=0.0614..0.647 rows=1000 loops=1)
-> Hash -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725
rows=1) (actual time=0.0276..0.0333 rows=1 loops=1)
-> Select #2 (subquery in projection; run only once)
-> Aggregate: avg(t2.Amount) (cost=202 rows=1) (actual time=1.14..1.14 rows=1 loops=1)
-> Inner hash join (t2.AccountNumber = b2.AccountNumber) (cost=102 rows=1000) (actual
time=0.0592..0.895 rows=1000 loops=1)
-> Table scan on t2 (cost=102 rows=1000) (actual time=0.0369..0.46 rows=1000 loops=1)
-> Hash -> Covering index lookup on b2 using i_bankaccount_user (UserID=256) (cost=0.725
rows=1) (actual time=0.0098..0.0114 rows=1 loops=1)
```

This index had no clear improvement from the original query. This is because the query aggregates using temporary tables so the grouping is broad. This makes the index ineffective.

Index #3:

```
CREATE INDEX i_taa ON Transactions(AccountNumber, Amount);

EXPLAIN ANALYZE
SELECT t.PaymentMethod, AVG(t.Amount) AS AvgTransaction,
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     JOIN BankAccount b2 ON t2.AccountNumber = b2.AccountNumber
     WHERE b2.UserID = 256) AS OverallAvg
FROM Transactions t
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
WHERE b.UserID = 256
GROUP BY t.PaymentMethod
```

```
ORDER BY AvgTransaction DESC;
```

Output:

```
-> Sort: AvgTransaction DESC (actual time=2.3..2.3 rows=5 loops=1)
-> Table scan on <temporary> (actual time=2.27..2.27 rows=5 loops=1)
-> Aggregate using temporary table (actual time=2.27..2.27 rows=5 loops=1)
-> Inner hash join (t.AccountNumber = b.AccountNumber) (cost=102 rows=1000) (actual time=0.105..1.13 rows=1000 loops=1)
-> Table scan on t (cost=102 rows=1000) (actual time=0.0504..0.641 rows=1000 loops=1)
-> Hash -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0274..0.0326 rows=1 loops=1)
-> Select #2 (subquery in projection; run only once)
-> Aggregate: avg(t2.Amount) (cost=205 rows=1) (actual time=2.66..2.66 rows=1 loops=1)
-> Nested loop inner join (cost=105 rows=1000) (actual time=0.392..2.46 rows=1000 loops=1)
-> Covering index lookup on b2 using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.11..0.111 rows=1 loops=1)
-> Index lookup on t2 using idx_transactions_acc_date (AccountNumber=b2.AccountNumber) (cost=104 rows=1000) (actual time=0.281..2.24 rows=1000 loops=1)
```

The performance of this index from the original query got a little worse. The cost stayed the same but the time increased by 1 millisecond consistently. The extra column, Amount, did not actually help the joins happening in the query. This caused some overhead.

Final Result: Index #1 was the best index because it helped the subquery and main query joins without adding any overhead or complexity.

Query 4: Last Few Transactions per Category

```
EXPLAIN ANALYZE
SELECT
    t.TransactionID,
    t.Date AS TransactionDate,
    t.Amount,
    t.PaymentMethod,
    c.CategoryName
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
LEFT JOIN Transactions t2
```

```
    JOIN TransactionCategory tc2
        ON t2.TransactionID = tc2.TransactionID
    ON tc.CategoryID = tc2.CategoryID
    AND b.UserID = 256 AND t.Date < t2.Date AND t.AccountNumber =
t2.AccountNumber
WHERE b.UserID = 256
GROUP BY t.TransactionID, c.CategoryName, t.Date, t.Amount,
t.PaymentMethod
HAVING COUNT(t2.TransactionID) < 5
ORDER BY c.CategoryName, t.Date DESC
LIMIT 15;
```

Output:

```
EXPLAIN -> Limit: 15 row(s) (actual time=501..501 rows=15 loops=1)
    -> Sort: c.CategoryName, t.`Date` DESC (actual time=501..501 rows=15 loops=1)
    -> Filter: (`count(t2.TransactionID)` < 5) (actual time=500..501 rows=59 loops=1)
    -> Table scan on <temporary> (actual time=500..500 rows=1000 loops=1)
    -> Aggregate using temporary table (actual time=500..500 rows=1000 loops=1)
    -> Nested loop left join (cost=10741 rows=100000) (actual time=0.195..369 rows=49630
    loops=1)
    -> Nested loop inner join (cost=455 rows=1000) (actual time=0.132..4.4 rows=1000 loops=1)
    -> Nested loop inner join (cost=105 rows=1000) (actual time=0.108..0.991 rows=1000 loops=1)
    -> Inner hash join (no condition) (cost=1.98 rows=10) (actual time=0.0715..0.138 rows=10
    loops=1)
    -> Table scan on c (cost=1.25 rows=10) (actual time=0.0198..0.0447 rows=10 loops=1)
    -> Hash -> Covering index lookup on b using UserID (UserID=256) (cost=0.725 rows=1) (actual
    time=0.0262..0.0314 rows=1 loops=1)
    -> Covering index lookup on tc using CategoryID (CategoryID=c.CategoryID) (cost=1.29
    rows=100) (actual time=0.0264..0.071 rows=100 loops=10)
    -> Filter: (t.AccountNumber = b.AccountNumber) (cost=0.25 rows=1) (actual
    time=0.00285..0.00302 rows=1 loops=1000)
    -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25
    rows=1) (actual time=0.00214..0.00219 rows=1 loops=1000)
    -> Nested loop inner join (cost=260 rows=100) (actual time=0.0418..0.357 rows=49.6
    loops=1000)
    -> Covering index lookup on tc2 using CategoryID (CategoryID=c.CategoryID) (cost=0.296
    rows=100) (actual time=0.0242..0.058 rows=101 loops=1000)
    -> Filter: ((t2.AccountNumber = b.AccountNumber) and (t.`Date` < t2.`Date`)) (cost=0.0025
    rows=1) (actual time=0.00259..0.00268 rows=0.491 loops=101096)
     -> Single-row index lookup on t2 using PRIMARY (TransactionID=tc2.TransactionID)
    (cost=0.0025 rows=1) (actual time=0.00178..0.00183 rows=1 loops=101096)
```

Index #1:

```sql
CREATE INDEX i_tad ON Transactions(AccountNumber, Date);

EXPLAIN ANALYZE
SELECT
    t.TransactionID,
    t.Date AS TransactionDate,
    t.Amount,
    t.PaymentMethod,
    c.CategoryName
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
LEFT JOIN Transactions t2
    JOIN TransactionCategory tc2
        ON t2.TransactionID = tc2.TransactionID
    ON tc.CategoryID = tc2.CategoryID
    AND b.UserID = 256 AND t.Date < t2.Date AND t.AccountNumber =
t2.AccountNumber
WHERE b.UserID = 256
GROUP BY t.TransactionID, c.CategoryName, t.Date, t.Amount,
t.PaymentMethod
HAVING COUNT(t2.TransactionID) < 5
ORDER BY c.CategoryName, t.Date DESC
LIMIT 15;
```

Output:

```
EXPLAIN -> Limit: 15 row(s) (actual time=472..472 rows=15 loops=1) -> Sort: c.CategoryName, t.`Date` DESC (actual time=472..472 rows=15 loops=1) -> Filter: (`count(t2.TransactionID)` < 5) (actual time=471..472 rows=59 loops=1) -> Table scan on <temporary> (actual time=471..472 rows=1000 loops=1) -> Aggregate using temporary table (actual time=471..471 rows=1000 loops=1) -> Nested loop left join (cost=10741 rows=100000) (actual time=0.108..346 rows=49630 loops=1) -> Nested loop inner join (cost=455 rows=1000) (actual time=0.0733..4.34 rows=1000 loops=1) -> Nested loop inner join (cost=105 rows=1000) (actual time=0.0636..0.89 rows=1000 loops=1) -> Inner hash join (no condition) (cost=1.98 rows=10) (actual time=0.0441..0.115 rows=10 loops=1) -> Table scan on c (cost=1.25 rows=10) (actual time=0.0203..0.0521 rows=10 loops=1) -> Hash -> Covering index lookup on b using
```

```
    i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0123..0.0143 rows=1
    loops=1) -> Covering index lookup on tc using i_tc_categoryid (CategoryID=c.CategoryID)
    (cost=1.29 rows=100) (actual time=0.0226..0.0639 rows=100 loops=10) -> Filter:
    (t.AccountNumber = b.AccountNumber) (cost=0.25 rows=1) (actual time=0.0029..0.00307
    rows=1 loops=1000) -> Single-row index lookup on t using PRIMARY
    (TransactionID=tc.TransactionID) (cost=0.25 rows=1) (actual time=0.00222..0.00228 rows=1
    loops=1000) -> Nested loop inner join (cost=260 rows=100) (actual time=0.042..0.333
    rows=49.6 loops=1000) -> Covering index lookup on tc2 using i_tc_categoryid
    (CategoryID=c.CategoryID) (cost=0.296 rows=100) (actual time=0.0253..0.0576 rows=101
    loops=1000) -> Filter: ((t2.AccountNumber = b.AccountNumber) and (t.`Date` < t2.`Date`))
    (cost=0.0025 rows=1) (actual time=0.00237..0.00244 rows=0.491 loops=101096) -> Single-row
    index lookup on t2 using PRIMARY (TransactionID=tc2.TransactionID) (cost=0.0025 rows=1)
    (actual time=0.00156..0.00162 rows=1 loops=101096)
```

This index had a decent improvement in performance. The cost stayed the same but the time improved around a millisecond consistently. The index reduces the temporary table use and results in fewer loop iterations.

Index #2:

```
CREATE INDEX i_tctid ON TransactionCategory(TransactionID);

EXPLAIN ANALYZE
SELECT
    t.TransactionID,
    t.Date AS TransactionDate,
    t.Amount,
    t.PaymentMethod,
    c.CategoryName
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
LEFT JOIN Transactions t2
    JOIN TransactionCategory tc2
        ON t2.TransactionID = tc2.TransactionID
    ON tc.CategoryID = tc2.CategoryID
    AND b.UserID = 256 AND t.Date < t2.Date AND t.AccountNumber =
t2.AccountNumber
WHERE b.UserID = 256
```

```
GROUP BY t.TransactionID, c.CategoryName, t.Date, t.Amount,
t.PaymentMethod
HAVING COUNT(t2.TransactionID) < 5
ORDER BY c.CategoryName, t.Date DESC
LIMIT 15;
```

Output:

```
EXPLAIN -> Limit: 15 row(s) (actual time=509..509 rows=15 loops=1) -> Sort: c.CategoryName, t.`Date` DESC (actual time=509..509 rows=15 loops=1) -> Filter: (`count(t2.TransactionID)` < 5) (actual time=508..509 rows=59 loops=1) -> Table scan on <temporary> (actual time=508..509 rows=1000 loops=1) -> Aggregate using temporary table (actual time=508..508 rows=1000 loops=1) -> Nested loop left join (cost=10741 rows=100000) (actual time=0.186..369 rows=49630 loops=1) -> Nested loop inner join (cost=455 rows=1000) (actual time=0.125..5.43 rows=1000 loops=1) -> Nested loop inner join (cost=105 rows=1000) (actual time=0.108..1.85 rows=1000 loops=1) -> Inner hash join (no condition) (cost=1.98 rows=10) (actual time=0.0688..0.146 rows=10 loops=1) -> Table scan on c (cost=1.25 rows=10) (actual time=0.0149..0.0456 rows=10 loops=1) -> Hash -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0319..0.0375 rows=1 loops=1) -> Covering index lookup on tc using i_tc_categoryid (CategoryID=c.CategoryID) (cost=1.29 rows=100) (actual time=0.112..0.157 rows=100 loops=10) -> Filter: (t.AccountNumber = b.AccountNumber) (cost=0.25 rows=1) (actual time=0.00302..0.00319 rows=1 loops=1000) -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25 rows=1) (actual time=0.0023..0.00235 rows=1 loops=1000) -> Nested loop inner join (cost=260 rows=100) (actual time=0.0432..0.356 rows=49.6 loops=1000) -> Covering index lookup on tc2 using i_tc_categoryid (CategoryID=c.CategoryID) (cost=0.296 rows=100) (actual time=0.0251..0.0586 rows=101 loops=1000) -> Filter: ((t2.AccountNumber = b.AccountNumber) and (t.`Date` < t2.`Date`)) (cost=0.0025 rows=1) (actual time=0.00257..0.00266 rows=0.491 loops=101096) -> Single-row index lookup on t2 using PRIMARY (TransactionID=tc2.TransactionID) (cost=0.0025 rows=1) (actual time=0.00174..0.00179 rows=1 loops=101096)
```

This index had no change in performance from the original query. There really is no filtering benefit for the TransactionID and this adds a minor overhead due to some redundancies.

Index #3:

```
CREATE INDEX i_tan ON Transactions(AccountNumber);

EXPLAIN ANALYZE
SELECT
    t.TransactionID,
    t.Date AS TransactionDate,
    t.Amount,
    t.PaymentMethod,
```

```
    c.CategoryName
FROM Transactions t
JOIN TransactionCategory tc
    ON t.TransactionID = tc.TransactionID
JOIN Category c
    ON tc.CategoryID = c.CategoryID
JOIN BankAccount b
    ON t.AccountNumber = b.AccountNumber
LEFT JOIN Transactions t2
    JOIN TransactionCategory tc2
        ON t2.TransactionID = tc2.TransactionID
    ON tc.CategoryID = tc2.CategoryID
    AND b.UserID = 256 AND t.Date < t2.Date AND t.AccountNumber =
t2.AccountNumber
WHERE b.UserID = 256
GROUP BY t.TransactionID, c.CategoryName, t.Date, t.Amount,
t.PaymentMethod
HAVING COUNT(t2.TransactionID) < 5
ORDER BY c.CategoryName, t.Date DESC
LIMIT 15;
```

Output:

```
EXPLAIN -> Limit: 15 row(s) (actual time=488..488 rows=15 loops=1) -> Sort: c.CategoryName, t.`Date` DESC (actual time=488..488 rows=15 loops=1) -> Filter: (`count(t2.TransactionID)` < 5) (actual time=487..488 rows=59 loops=1) -> Table scan on <temporary> (actual time=487..488 rows=1000 loops=1) -> Aggregate using temporary table (actual time=487..487 rows=1000 loops=1) -> Nested loop left join (cost=10741 rows=100000) (actual time=0.195..358 rows=49630 loops=1) -> Nested loop inner join (cost=455 rows=1000) (actual time=0.127..4.72 rows=1000 loops=1) -> Nested loop inner join (cost=105 rows=1000) (actual time=0.11..1.06 rows=1000 loops=1) -> Inner hash join (no condition) (cost=1.98 rows=10) (actual time=0.0637..0.146 rows=10 loops=1) -> Table scan on c (cost=1.25 rows=10) (actual time=0.017..0.0512 rows=10 loops=1) -> Hash -> Covering index lookup on b using i_bankaccount_user (UserID=256) (cost=0.725 rows=1) (actual time=0.0249..0.029 rows=1 loops=1) -> Covering index lookup on tc using i_tc_categoryid (CategoryID=c.CategoryID) (cost=1.29 rows=100) (actual time=0.0261..0.0769 rows=100 loops=10) -> Filter: (t.AccountNumber = b.AccountNumber) (cost=0.25 rows=1) (actual time=0.00307..0.00324 rows=1 loops=1000) -> Single-row index lookup on t using PRIMARY (TransactionID=tc.TransactionID) (cost=0.25 rows=1) (actual time=0.00235..0.0024 rows=1 loops=1000) -> Nested loop inner join (cost=260 rows=100) (actual time=0.0437..0.346 rows=49.6 loops=1000) -> Covering index lookup on tc2 using i_tc_categoryid (CategoryID=c.CategoryID) (cost=0.296 rows=100) (actual time=0.0247..0.0581 rows=101 loops=1000) -> Filter: ((t2.AccountNumber = b.AccountNumber) and (t.`Date` < t2.`Date`))
```

(cost=0.0025 rows=1) (actual time=0.00248..0.00256 rows=0.491 loops=101096) -> Single-row index lookup on t2 using PRIMARY (TransactionID=tc2.TransactionID) (cost=0.0025 rows=1) (actual time=0.00166..0.00172 rows=1 loops=101096)

This index saw no improvement in performance from the original query. This index does help the AccountNumber join but there is still no ordering of the Date attribute. Therefore, the query still scans the same amount of rows and transactions.

Final Result: Index #1 is the best index because it reduces the nested loops needed to execute the query, and it also reduces redundant or range lookups inside of the temporary tables.

Final Results Table

Query	Best Index	Cost Before	Cost After	Improvement %
Q1	1st Index	336	207	38%
Q2	1st Index	235	230	2%
Q3	1st Index	105	102	3%
Q4	1st Index	10741	10741	0%